

Memoria

-

Trabajo webapp en React 2026



Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Ruben Cameo
Alejandro Guerra

1. Introducción.....	4
1.1. Objeto del proyecto.....	4
1.2. Tecnologías utilizadas.....	4
1.3. Servicios cloud.....	4
2. Arquitectura de la aplicación.....	5
2.1. Arquitectura hexagonal.....	5
2.2. Organización del proyecto.....	5
2.2.1. domain.....	5
2.2.1.1. Pelicula.tsx.....	6
2.2.1.2. Usuario.tsx.....	9
2.2.1.3. Home.tsx.....	10
2.2.1.4. ui.tsx.....	10
2.2.1.5. Busqueda.tsx.....	10
2.2.1.6. Header.tsx.....	11
2.2.2. infrastructure.....	11
2.2.2.1. AccessRepository.tsx.....	11
2.2.2.2. PeliculaRepository.tsx.....	13
2.2.3. services.....	14
2.2.3.1. AuthService.tsx.....	14
2.2.4. pages.....	15
2.2.4.1. AvisoLegal.tsx.....	15
2.2.4.2. Contacto.tsx.....	16
2.2.4.3. detallepelicula.css y DetallePelicula.tsx.....	17
2.2.4.4. Favoritos.tsx.....	19
2.2.4.5. home.css y Home.tsx.....	19
2.2.4.6. TopPeliculas.....	22
2.2.5. components.....	23
2.2.5.1. access.....	23
a) Login.tsx.....	23
b) Registro.tsx.....	24
2.2.5.2. ui.....	25
a) CarouselPrincipal.tsx.....	25
b) footer.css y Footer.tsx.....	26
c) Header.tsx y header.css.....	27
d) MensajeModal.tsx.....	28
2.2.6. store.....	28
2.2.6.1. AuthContext.tsx.....	28
2.2.7. utils.....	29
3. Modelo de datos.....	30
3.1. Firebase Authentication.....	30
3.2. Estructura de la Realtime Database.....	30
3.2.1. Nodo de películas.....	30
3.2.2. Nodo de usuarios.....	31
3.2.3. Nodo de puntuaciones.....	31
3.2.4. Nodo de comentarios.....	31
4. Flujo de usuario y funcionalidades.....	32

4.1. Gestión de sesiones (access).....	32
4.2. Experiencia de visualización.....	32
4.3. Interactividad.....	32
4.4. Personalización.....	32
5. Interfaz y experiencia de usuario.....	33
5.1. Diseño visual.....	33
5.2. Componentes globales.....	33
5.3. Feedback y modales.....	33
6. Formato móvil (Responsive).....	34
7. Mejoras introducidas para alcanzar los 2.5 puntos.....	39
7.1. Indicar las películas mejor valoradas.....	39
7.2. Mejores notificaciones que un simple alert.....	40
7.3. Posibilidad de implementar diferentes idiomas.....	41
7.4. Uso de librería para introducir mapa.....	43
Detalles de la implementación.....	43
7.5. Implementación de triangulo + estrella para indicar película favorita.....	44
7.6. Implementación de búsqueda.....	44
7.7. Opción de ver la película (que aparezca) aunque no se pueda poner por derechos.....	45
7.8. Mejora en la visualización de las Cards de las películas.....	46
Lógica de Internacionalización Dinámica.....	46
8. Metodología y Control de versiones.....	48
8.1. Gestión del repositorio.....	48
8.2. Historial de commits.....	48
8.3. Despliegue (Hosting).....	48
9. Consecución de los objetivos.....	50
10. Conclusiones y posibles mejoras.....	51

1. Introducción

1.1. Objeto del proyecto

Este proyecto consiste en crear una web de películas donde la gente pueda entrar a ver un catálogo completo, como si fuera un videoclub online. La idea es que la web sea interactiva: que no solo sirva para leer información, sino que los usuarios puedan crearse una cuenta, entrar con su correo, dejar sus propios comentarios y poner notas a las pelis que han visto.

Además, hemos añadido una función para que cada usuario tenga su propia lista de Favoritos, donde pueda guardar lo que más le gusta y gestionarlo (añadir o borrar) cuando quiera.

Para completar la experiencia, la plataforma incluye secciones de información corporativa y legal, junto con un ranking dinámico que destaca las películas mejor valoradas por la comunidad

1.2. Tecnologías utilizadas

Para el desarrollo de la página hemos usado:

Tecnología	Explicación
React	La librería principal que usamos para organizar la web por componentes.
TypeScript	Nos sirve para definir el tipo de datos que manejamos y evitar errores en el código.
Bootstrap	Librería de estilos que usamos para que el diseño sea ordenado, los botones funcionen bien y la web se adapte a móviles.
Axios	Herramienta para conectar la aplicación con la base de datos y poder pedir o enviar información.

Tabla 1. Conjunto de tecnologías empleadas

1.3. Servicios cloud

Para garantizar la escalabilidad y disponibilidad de la plataforma, hemos integrado Google Firebase como proveedor de servicios en la nube, utilizando los siguientes módulos:

	Explicación
Autenticación	Gestiona el registro de nuevos usuarios y el inicio de sesión.
Realtime Database	Es la base de datos donde se almacena el catálogo de películas, las reseñas de los usuarios y sus listas personales.
Hosting	Servicio de alojamiento donde hemos subido el proyecto para que esté disponible online a través de una URL.

Tabla 2. Apartados de Firebase empleados

2. Arquitectura de la aplicación

2.1. Arquitectura hexagonal

Para el desarrollo de esta webapp se ha empleado una arquitectura hexagonal (también conocida como de Puertos y Adaptadores). Se ha elegido este patrón debido a que es el estándar en la industria para lograr un desacoplamiento efectivo entre la lógica de negocio y los agentes externos. Al aislar las funcionalidades principales de la aplicación de las herramientas de infraestructura (como Firebase para el backend o Axios para las peticiones HTTP), garantizamos que el código sea más mantenible y escalable.

2.2. Organización del proyecto

El código de la aplicación se centraliza en la carpeta principal src. Para garantizar un desarrollo escalable y facilitar el mantenimiento, el proyecto se ha organizado en 8 subcarpetas siguiendo los principios de la Arquitectura Hexagonal y la separación de responsabilidades.

Esta estructura permite aislar la lógica de negocio de la interfaz de usuario y de las dependencias externas (como Firebase o las APIs). A continuación, en la siguiente figura, se muestra el árbol de directorios resultante:

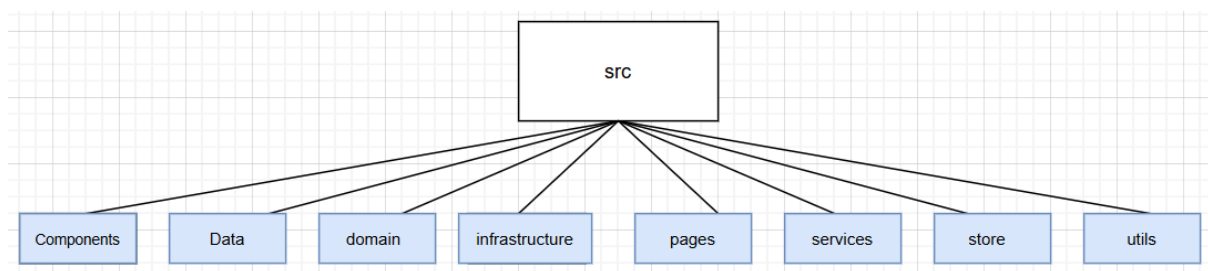


Fig 1. Estructura jerárquica de directorios en el núcleo del proyecto (src).

2.2.1. domain

En esta capa es donde definimos las interfaces y tipos de datos que representan los conceptos de nuestro sistema. Al trabajar con TypeScript, aprovechamos el tipado para garantizar que los datos (como Pelicula o Usuario) mantengan siempre la estructura correcta en toda la aplicación.

Al seguir una arquitectura hexagonal, esta capa es totalmente independiente de tecnologías externas.

Para organizar la lógica de negocio, hemos segmentado el dominio en varios archivos que aseguran la consistencia y seguridad de los datos:

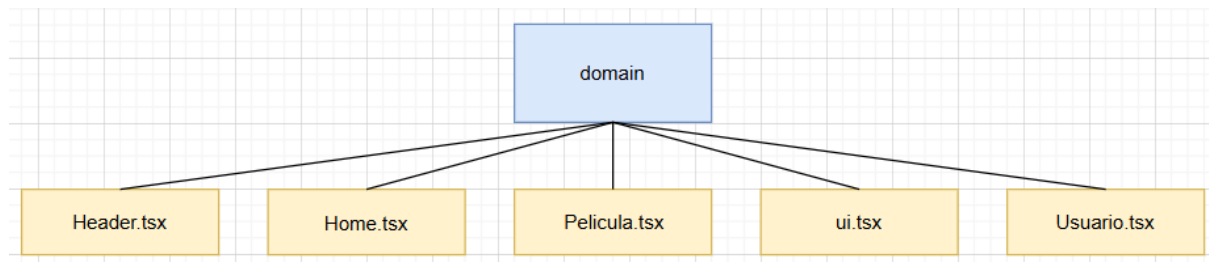


Fig 2. Estructura de modelos de datos en la capa de Dominio.

2.2.1.1. Pelicula.tsx

Este archivo contiene las estructuras necesarias para manejar el catálogo, los favoritos y las respuestas de Firebase.

Modelo Principal de Dominio

La interfaz Pelicula representa el objeto completo con el que trabaja la aplicación. Incluye tanto metadatos técnicos como los objetos de traducción.

Pelicula.tsx	Atributo	tipo
Película	id	number
	calificacion_media	number
	imagen_portada	string
	imagen_en_pelicula	string
	video_local	string
	fecha_estreno	string
	proximamente	boolean
	taquilla	number
	pais_origen	string
	mercados	any[]
	es / en /eu	InfoIntroducida

Tabla 3. Definición de la interfaz Pelicula en la capa de dominio

Soporte para Internacionalización

Para gestionar el contenido en diferentes idiomas, se ha definido la interfaz `InfoIntroducida`. Esta estructura agrupa los campos de texto que requieren traducción, permitiendo que la interfaz principal de Pelicula contenga un objeto específico para cada idioma (es, en, eu).

User.tsx	Atributo	tipo
InfoIntroducida	titulo	string
	categoria	string
	descripcion	string

Tabla 4. Definición de la interfaz `InfoIntroducida`

Gestión de Favoritos

Para optimizar el rendimiento, no siempre enviamos el objeto `Pelicula` completo. En el caso de los favoritos, usamos una versión simplificada (llamada `PeliFav`) que solo contiene lo básico.

Pelicula.tsx	Atributo	tipo
PeliFav	id	string
	titulo	string
	imagen_portada	string
	categoria	string
	pelicula_id	number

Tabla 5. Definición de la interfaz `PeliFav`

Definición de propiedades de la interfaz

Para que los componentes visuales reciban la información de forma segura y tipada, se utiliza la interfaz `Props`.

Pelicula.tsx	Atributo	tipo
Props	peliculas	<code>PeliculaFirebase[]</code>

Tabla 6. Definición de la interfaz `Props`

Modelos para componentes de presentación

Para optimizar el rendimiento del código, se han creado interfaces simplificadas que contienen la información necesaria para ciertos elementos visuales. La interfaz `Pelicula_destacada` es el contrato de datos utilizado por el carrusel principal de la aplicación.

Pelicula.tsx	Atributo	tipo
Pelicula_destacada	titulo	string
	imagen	string
	descripcion	string

Tabla 7. Definición de la interfaz `Pelicula_destacada`

Interfaces de Infraestructura (Firebase)

Estas estructuras se encargan de adaptar la forma en que Firebase nos entrega los datos para que encajen con nuestros modelos de TypeScript.

Pelicula.tsx	Atributo	tipo
FirebasePelicula	key	[key: string]: Omit<Pelicula, "id">

Tabla 8. Definición de la interfaz `FirebasePelicula`

Hay que recordar que Firebase Realtime Database guarda los datos como un gran objeto donde cada película está dependiente de una clave larga y rara (su ID). Al usar `Omit<Pelicula, "id">`, le decimos a TypeScript que el objeto que viene de Firebase trae todo lo que define a una película, menos el ID, ya que el ID es la propia clave de Firebase.

Modelo de Transformación

Finalmente, utilizamos una estructura intermedia para procesar los datos antes de convertirlos al modelo final de la aplicación:

Pelicula.tsx	Atributo	tipo
PeliculaFirebase	id	number
	titulo	string
	key	string

Tabla 9. Definición de la interfaz `PeliculaFirebase`

2.2.1.2. Usuario.tsx

En este apartado se definen los contratos necesarios para gestionar la autenticación y el perfil del usuario en toda la aplicación.

En primer lugar se encuentra `UserAuthResponse`, que define la estructura de la respuesta inmediata que recibimos de Firebase tras una autenticación exitosa.

Usuario.tsx	Atributo	tipo
UserAuthResponse	idToken	string
	localId	string
	email	string

Tabla 10. Interfaz de `UserAuthResponse`

Gestión del perfil de usuario

Representa la información que almacenamos de cada persona que se registra en nuestra plataforma.

Pelicula.tsx	Atributo	tipo
UserProfile	user	string
	email	string
	fecha_registro	string

Tabla 11. Definición de la interfaz `UserProfile`

El Contrato de Sesión

Para que toda la web sepa si hay alguien logueado, usamos un Contexto de React que actúa como un contrato global. Esto permite que cualquier botón o página sepa quién es el usuario actual y qué puede hacer.

ui.tsx	Atributo	tipo
AuthContextType	login	string
	language	string
	idToken	string
	userID	string
	userName	string
	loginAction	void
	logoutAction	void

Tabla 12. Interfaz de `AuthContextType`

2.2.1.3. Home.tsx

Aquí se define la interfaz HomeProps, que actúa como el contrato de datos para el componente. Solo contiene textobuscado, que es el string que recibimos desde la barra de navegación para que la pantalla principal sepa qué resultados filtrar y mostrar.

2.2.1.4. ui.tsx

En este caso se exporta la interfaz llamada AvisoDeProps, que define la estructura necesaria para controlar los mensajes modales de la aplicación:

ui.tsx	Atributo	tipo
AvisoDeProps	show	boolean
	onHide()	void
	titulo	string
	mensaje	string
	tipo	'success' 'error'

Tabla 13. Interfaz de AvisoDeProps

Como se observa en la estructura anterior, el atributo show actúa como un booleano que determina la visibilidad del componente en la interfaz. Por su parte, la función onHide es la encargada de gestionar el cierre del aviso. Finalmente, el componente se construye de forma dinámica recibiendo el título, el cuerpo del mensaje y el tipo de alerta.

2.2.1.5. Busqueda.tsx

Este componente es el encargado de gestionar la experiencia del usuario cuando intenta localizar contenido específico dentro del catálogo.

ui.tsx	Atributo	tipo
ResultadosBusquedaProps	películas	Pelicula[]
	textointroducido	string

Tabla 14. Interfaz de ResultadosBusquedaProps

Para que el componente funcione correctamente, es necesario inyectar tanto el catálogo completo de películas como el valor actual del buscador.

2.2.1.6. Header.tsx

En este caso, lo único que se tiene que introducir es la función `onSearch`. Al definirla aquí, estamos obligando a que cualquier cabecera que usemos en el futuro sea capaz de capturar el texto que escribe el usuario y enviarlo a la lógica de filtrado.

ui.tsx	Atributo	tipo
HeaderProps	onSearch	void

Tabla 15. Interfaz de `AvisoDeProps`

2.2.2. infrastructure

Esta capa es la encargada de gestionar la comunicación entre nuestra aplicación y los servicios externos. En nuestro caso, actúa como el adaptador que conecta la lógica de negocio con la API de Firebase.

2.2.2.1. AccessRepository.tsx

Este repositorio centraliza toda la lógica de autenticación y acceso a la plataforma. Utiliza los servicios de Google Identity Toolkit para la gestión de credenciales y Firebase Realtime Database para el almacenamiento de perfiles. El repositorio se apoya en constantes externas como la `API_KEY`, `AUTH_URL` y base de datos (`DB_URL`) para realizar peticiones.

Funcion	Parámetros de entrada	
	Atributo	tipo
login	email	string
	pass	string
registroCompleto	email	string
	pass	string
	username	string
obtenerNombreUsuario	localId	string
	idToken	string

Tabla 16. Conjunto de funciones definidas en `AccessRepository`

Validación de Credenciales (Login)

El método `login` permite a los usuarios ya registrados acceder a la aplicación. Envía el correo electrónico (`email`) y la contraseña (`pass`). Se incluye el parámetro `returnSecureToken`, lo que fuerza al servidor a devolver un token de acceso si las credenciales son correctas, permitiendo así mantener la sesión activa.

Proceso de Registro en Dos Pasos

La función `registroCompleto` gestiona la creación de nuevas cuentas mediante un flujo combinado. En primer lugar, registra al usuario en el sistema de identidades de Google. Una vez obtenido el identificador único (`localId`), el método genera un objeto de tipo `UserProfile` que incluye el nombre de usuario elegido y la fecha de registro actual. Finalmente, guarda esta información en un nodo de la base de datos, vinculando la identidad de autenticación con los datos personales del usuario.

Recuperación de Información de Perfil

Mediante el método `obtenerNombreUsuario`, el repositorio permite consultar el nombre de perfil asociado a un usuario concreto. Esta función realiza una petición GET autenticada mediante el `idToken` al nodo del usuario. En caso de que no exista un nombre personalizado en la base de datos, el sistema devuelve por defecto el valor "Usuario", garantizando que la interfaz siempre tenga un nombre que mostrar en la barra de navegación o el perfil.

2.2.2.2. PeliculaRepository.tsx

Este repositorio es el más extenso de la capa de infraestructura, ya que gestiona todo el flujo de información del catálogo, la interacción con el usuario y el almacenamiento de las listas personales. Su función principal es actuar como puente entre la aplicación y Firebase Realtime Database. A continuación, se detallan las funcionalidades implementadas:

Funcion	Atributo	tipo
getAll	-	-
getById	id	string
getComentarios	pelicula_id	string
saveComentario	nuevoComentario	any
getPuntuaciones	pelicula_id	string
savePuntuacion	nuevaPuntuacion	any
saveFavoritos	userId	string
	favorito	any
getFavoritos	userId	string
	token	string
deleteFavorito	userId	string
	pelid	string
	token	string
getFavoritoById	userId	string
	token	string
getTopRanking	-	-
getDestacadas	-	-
getAvailableMovies	-	-

Tabla 17. Catálogo de métodos del repositorio de películas

Gestión del Catálogo de Películas

Para la obtención de información, el repositorio implementa el método `getAll`, que descarga el catálogo completo y transforma el objeto de Firebase en un array de objetos tipo `Pelicula`. Complementariamente, `getById` permite recuperar la información detallada de una única película mediante su id. Para la página principal y filtros, se han desarrollado `getTopRanking` (que ordena las películas por su nota media), `getDestacadas` (que filtra aquellas marcadas como relevantes) y `getAvailableMovies`, que asegura que solo se muestren películas con contenido traducido disponible.

Interacción y Feedback del Usuario

En cuanto a la interacción social, el repositorio gestiona los comentarios y las valoraciones. Mediante `getComentarios` y `saveComentario`, la aplicación puede listar y publicar opiniones filtradas por película. De igual forma, los métodos `getPuntuaciones` y `savePuntuacion` se encargan de gestionar el sistema de votos, permitiendo descargar todas las notas de una película para calcular su promedio en tiempo real o registrar una nueva puntuación vinculada al ID del usuario.

Listas Personales

Finalmente, el repositorio administra la sección de favoritos del usuario mediante peticiones autenticadas. El método `saveFavoritos` añade nuevas películas a la lista personal, mientras que `deleteFavorito` se encarga de su eliminación. Para la visualización, se utiliza `getFavoritos`, que mapea los datos de Firebase a un formato usable por la interfaz. Por último, se incluye la función `getFavoritoById`, diseñada específicamente para recuperar de forma ligera solo los IDs de las películas favoritas; esto permite que los botones de la interfaz sepan si una película ya está en la lista del usuario o no.

2.2.3. services

2.2.3.1. AuthService.tsx

Este archivo funciona como un traductor de excepciones. Cuando ocurre un problema durante el login o el registro, Firebase devuelve códigos de error técnicos en inglés que resultarían confusos para un usuario final. El propósito de este servicio es interceptar esos códigos y devolver un mensaje más amigable.

El servicio implementa el método `obtenerMensajeError`, el cual recibe un código de error (string) y, mediante una estructura de control switch, determina qué traducción debe mostrarse:

Funcion	tipo	mensaje
obtenerMensajeError	EMAIL_EXISTS	El correo ya está en uso. Prueba con otro o haz login
	WEAK_PASSWORD	La contraseña es muy corta. Pon al menos 6 caracteres.
	INVALID_EMAIL	El formato del correo no es correcto (falta @ o punto).
	OPERATION_NOT_ALLOWED	El registro con contraseña está desactivado en Firebase.

Tabla 18. Mapeo de códigos de error técnicos a mensajes de usuario localizados.

2.2.4. pages

Esta carpeta la empleamos para almacenar los componentes que representan cada una de las páginas o rutas de nuestra aplicación.

2.2.4.1. AvisoLegal.tsx

Este componente es una página estática cuyo objetivo es proporcionar transparencia informativa sobre el proyecto. Su implementación se basa en los siguientes puntos:

- **Finalidad Académica:**
Se especifica que la web es un entorno de pruebas desarrollado por nosotros para fines educativos.
- **Gestión de Derechos:**
Se aclara que el contenido multimedia pertenece a sus respectivos autores (no a nosotros), justificando su uso bajo el concepto de fines docentes.
- **Estructura Visual:**
Se emplea el sistema de rejilla de React-Bootstrap (Container, Row, Col) para garantizar una disposición fluida. La jerarquía se ha cuidado mediante el uso de etiquetas <section> para separar los bloques, empleando encabezados <h3> para los títulos y etiquetas <p> para el cuerpo del texto.



Fig 3. Captura de visualización de la página de Aviso Legal

2.2.4.2. Contacto.tsx

En esta sección, nuestra intención era ofrecer una página de contacto que fuera más allá de un simple formulario, dándole un toque más profesional:

- **Mapa dinámico con Leaflet:**

En lugar de una imagen fija, hemos integrado la librería React-Leaflet. Configuramos un mapa interactivo centrado en Pamplona donde situamos tres sedes mediante marcadores. Para que el código fuera limpio, creamos un array de sedes y lo recorremos con un `.map()`, lo que nos permite añadir o quitar puntos de venta (`<Marker>` y `<Popup>`).

- **Cabecera y Estilo Visual:**

Para la cabecera, hemos optado por una estética cinematográfica que simula el cartel de una película mediante una imagen de fondo a gran escala. Con el fin de garantizar la legibilidad del texto, aplicamos un degradado oscuro (linear-gradient) sobre la imagen.

- **Organización de la información:**

Usando el sistema de rejilla de Bootstrap, hemos distribuido los datos de contacto en tarjetas. Cada una incluye un icono temático y una combinación de colores turquesa y gris oscuro que define la identidad visual que hemos elegido para el proyecto. El primero de ellos lo establecemos para indicar el email de contacto. La segunda de las tarjetas la empleamos para indicar el teléfono de contacto. Por último, la tercera de las tarjetas es donde se muestra la dirección.

- **Adaptabilidad (Responsive):**

Nos hemos asegurado de que la página funcione bien en cualquier dispositivo. En móviles, las tarjetas de contacto y el mapa se ajustan automáticamente para ocupar todo el ancho, garantizando una buena experiencia de usuario.

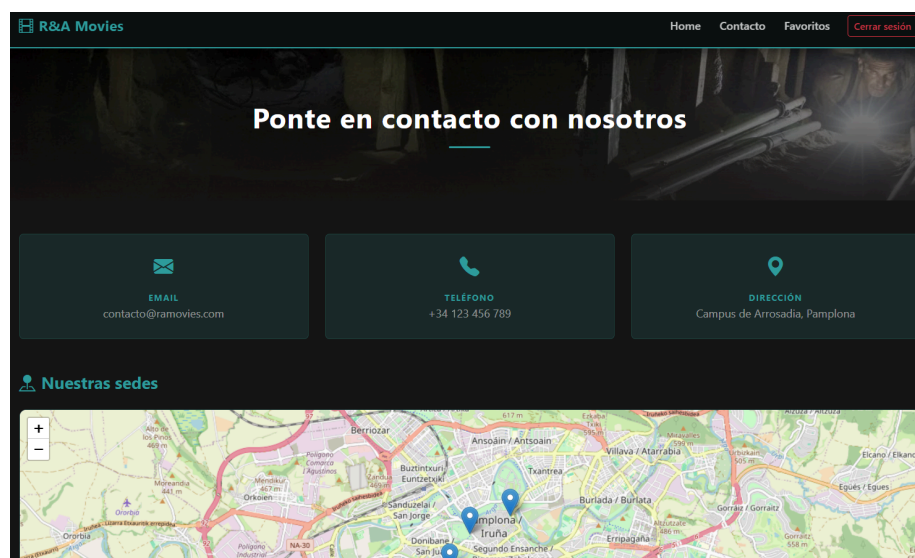


Fig 4. Captura de visualización de la página de Contacto

2.2.4.3. detallepelicula.css y DetallePelicula.tsx

Esta página es el núcleo interactivo de la aplicación. En ella, hemos implementado una lógica que permite al usuario no solo consultar información, sino participar activamente en la comunidad de la película.

a) detallepelicula.css

En la hoja de estilos hemos configurado el apartado de comentarios limitando su altura máxima a 300px para evitar que la página se alargue demasiado, activando un scroll vertical automático cuando el contenido supera ese espacio. Para que visualmente no desentone, hemos personalizado la barra de navegación dándole una anchura mínima de 5px, dejando el fondo del carril totalmente transparente y aplicando un color turquesa con bordes redondeados al tirador para que sea funcional pero discreto.

b) DetallePelicula.tsx

Para esta página, nos hemos centrado en la gestión de estados complejos y el uso de efectos:

- **Carga de datos en paralelo:**

Para optimizar el tiempo de respuesta, hemos utilizado Promise.all. De esta forma, lanzamos simultáneamente las peticiones para obtener los detalles de la película, los comentarios, las puntuaciones y el estado de favoritos, reduciendo el tiempo de carga inicial.

- **Sistema de Valoración:**

Hemos desarrollado una lógica que calcula el promedio de notas de la película en tiempo real (procesarPuntuaciones). Además, para evitar duplicados, hemos incluido una validación que comprueba si el usuario actual ya ha puntuado la película, bloqueando el formulario si es necesario.

- **Gestión de Favoritos:**

Gracias al uso del AuthContext, la página sabe si la película ya está en la lista personal del usuario. Si es así, el botón cambia automáticamente de color y texto mediante un sistema de "flags" (banderas) que hemos programado en el renderizado.

- **Protección de Contenido:**

Hemos condicionado el acceso a ciertas funciones. Si un usuario no está logueado, en lugar de los botones de reproducción o el formulario de comentarios, le mostramos un Badge informativo que le invita a iniciar sesión.

- **Feedback al Usuario:**

Para que la web sea profesional, hemos integrado un componente llamado MensajeModal. En lugar de usar alertas de navegador, informamos al usuario de que su comentario o voto se ha guardado correctamente mediante una ventana modal personalizada.

- **Maquetado:**

Hemos montado primero un bloque arriba del todo que ocupa gran parte de la pantalla para poner la foto de la película de fondo. Encima de esa imagen hemos colocado el título, las estrellas de la puntuación y los botones de "Ver ahora" y "Favoritos" para que sea lo primero que se vea. Después, hemos abierto otro bloque más abajo dividido en dos columnas: en la izquierda hemos metido el texto de la sinopsis y el formulario para escribir opiniones, y en la derecha hemos encajado el vídeo del tráiler. Por último, abajo del todo, el código revisa si la película tiene comentarios en la base de datos; si los tiene, los va pintando uno a uno con el nombre del usuario, y si no hay ninguno, simplemente saca un aviso diciendo que todavía no hay reseñas.

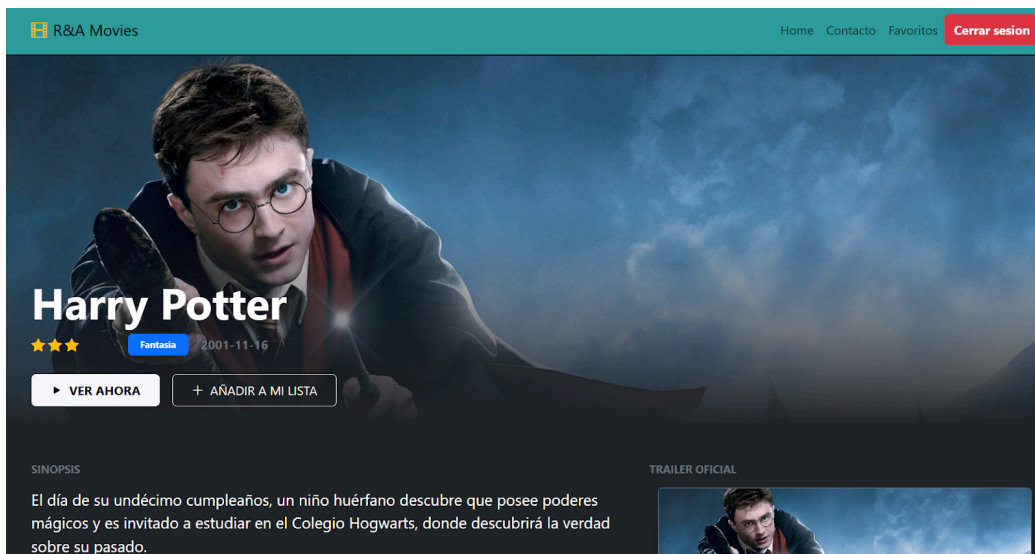


Fig 5. Visualización de la página con los detalles de una película (1)

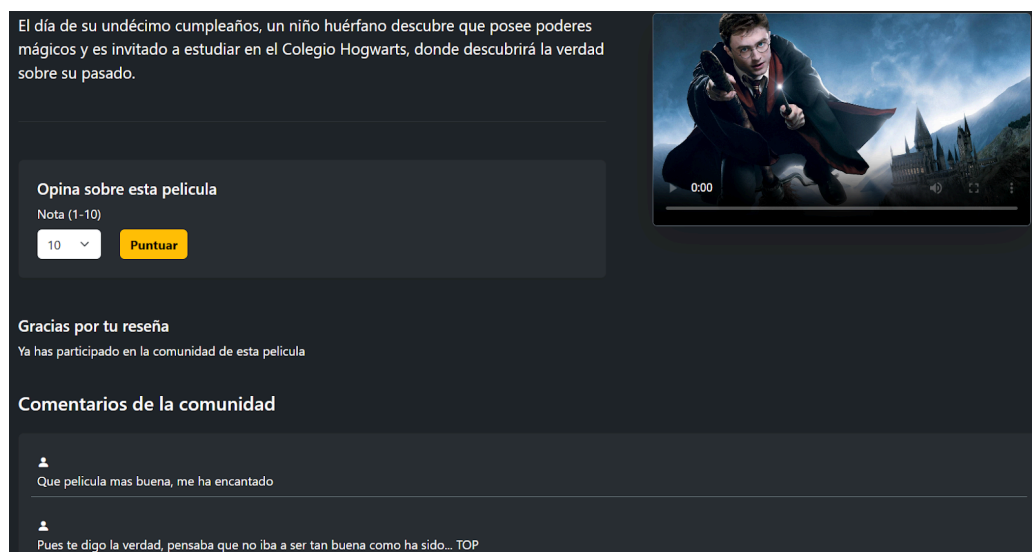


Fig 6. Visualización de la página con los detalles de una película (2)

2.2.4.4. Favoritos.tsx

Esta página funciona como la biblioteca personal del usuario. Hemos diseñado esta vista para que sea un espacio privado donde cada usuario pueda gestionar las películas que ha guardado previamente.

- **Persistencia y Seguridad:**

Para cargar esta página, hemos integrado el AuthContext de forma que solo intentamos recuperar datos si el userId y el token están presentes. Esto garantiza que un usuario no pueda ver la lista de otro y que las peticiones a Firebase estén siempre autorizadas.

- **Gestión Dinámica de la Lista:**

Uno de los puntos clave que hemos implementado es el eliminarFavoritoHandler. Al borrar una película, no solo enviamos la orden a la base de datos, sino que actualizamos el estado de React inmediatamente usando un .filter(). Esto permite que la película desaparezca de la pantalla al instante sin necesidad de recargar la página. Lo que hacemos en específico es filtrar la lista actual para quedarnos con todas las películas excepto la que acabamos de eliminar

- **Interfaz Basada en Tarjetas (Cards):**

Hemos utilizado un sistema de rejilla adaptable (xs, sm, md, lg) para que las carátulas se ajusten perfectamente a cualquier tamaño de pantalla. Cada tarjeta se estructura mediante un contenedor principal que muestra la imagen de la película en la parte superior. Justo debajo, incluimos el título y un sistema de filas y columnas para organizar las acciones. En este espacio, implementamos un enlace (<Link>) que permite navegar al detalle de la película y, además, añadimos un botón específico para eliminarla de la lista de favoritos de forma directa.

- **Mensajes:**

Además, hemos vuelto a utilizar nuestro componente MensajeModal para confirmar visualmente al usuario cuando una película ha sido eliminada con éxito.

2.2.4.5. home.css y Home.tsx

La Home es el componente más dinámico de nuestra web. En ella, hemos integrado el catálogo completo de Firebase y desarrollado un sistema de navegación interna para que el usuario encuentre fácilmente lo que busca.

a) home.css

Para el apartado visual, nuestro objetivo era alejarnos del aspecto estándar de Bootstrap y conseguir una estética de plataforma de streaming:

- **Diseño de Cards (formato netflix):**

Para lograr el efecto de catálogo dinámico, hemos configurado la clase principal de la tarjeta con position: relative y overflow: hidden, lo que permite que cualquier elemento que sobresalga al ampliar la imagen quede recortado dentro de los bordes. Hemos aplicado un z-index: 1 para establecer una base de capas y una transición suave (transition) que gestiona cómo cambia el tamaño de la tarjeta al interactuar con ella.

Al activar el estado `:hover`, el navegador ejecuta una transformación de `scale(1.07)`, haciendo que la miniatura crezca ligeramente hacia el usuario para resaltar la selección. En el apartado de la imagen (`netflix-card-img`), hemos usado `object-fit: cover` para asegurar que las carátulas se adapten al contenedor sin deformarse. Por último, al pasar el ratón, ajustamos la `opacity: 0` de la barra de título original para que el diseño se limpie y el usuario pueda centrarse exclusivamente en la imagen de la película que ha seleccionado.

Después hemos usado las clases `netflix-title-bar-text` y `netflix-title-bar-cat` para organizar la información que aparece bajo la imagen. Con el primero le damos un color blanco y un grosor de fuente al título para que sea lo que más destaque, mientras que con el segundo aplicamos un color turquesa, un tamaño más pequeño y letras en mayúsculas a la categoría. De esta forma, conseguimos una jerarquía visual clara.

Para que la tarjeta sea interactiva, hemos diseñado la clase `netflix-hover-overlay`. Se trata de una capa con `position: absolute` que cubre toda la tarjeta y utiliza un degradado oscuro (`linear-gradient`) para que el texto blanco se lea perfectamente. Por defecto, esta capa es invisible (`opacity: 0`) y está desplazada un poco hacia abajo (`translateY(8px)`), pero al pasar el ratón sobre la tarjeta, se vuelve opaca y sube suavemente a su posición original mediante una transición.

Dentro de esta capa, usamos `flex-direction: column` para organizar verticalmente el contenido: el título resaltado, la categoría en mayúsculas, una breve descripción y los metadatos de la película. Además, hemos incluido el botón `netflix-btn-detalles`, que tiene un diseño rectangular gracias al ajuste de sus rellenos (`padding`) y un color turquesa que cambia de tono y se eleva ligeramente al interactuar con él, indicando al usuario que puede hacer clic para ver más información.

- **Parte de películas favoritas:**

Para indicar que una película es favorita, hemos creado una etiqueta llamada `favorito-ribbon` que aparece sobre la tarjeta. Al ponerle `position: absolute` y marcarle `top: 0` y `right: 0`, conseguimos que el aviso se quede fijo en la esquina de arriba a la derecha sin mover el resto de elementos. El efecto de triángulo lo hacemos jugando con los bordes: dejamos tres lados transparentes y solo le damos color al que nos interesa con el tono verde. Por último, le hemos asignado un `z-index: 10` para asegurarnos de que la etiqueta siempre se vea por encima de la imagen de la película.

Por otro lado, definimos el estilo para la etiqueta `i` dentro de `favorito-ribbon`, lo que nos permite controlar el icono que va dentro de la etiqueta. Lo configuramos con `position: absolute` y lo ajustamos con `top: 5px` y `right: -40px` para que quede perfectamente centrado dentro del triángulo verde. Finalmente, le aplicamos un color blanco y un tamaño de fuente pequeño.

b) Home.tsx

En el desarrollo de este componente, nos hemos centrado en la organización inteligente de la información mediante el uso de estados de React:

- **Categorización de contenidos:**

En lugar de mostrar todas las películas en una lista interminable, hemos programado tres filtros principales: "Cartelera", "Próximamente" y "Catálogo Completo". Usamos un estado (seccionActiva) para decidir qué conjunto de datos mostrar en cada momento.

- **Filtrado por Género:**

Dentro del catálogo, hemos implementado un sistema de filtrado por categorías (Acción, Drama, etc.). Al pulsar un botón, la aplicación filtra el array de películas en tiempo real, permitiendo una navegación muy fluida sin recargas.

- **Componentes Reutilizables:**

Para no repetir código, usamos la función renderTarjetasPelicula, que sirve para pintar filas de pelis en cualquier parte de la web. Esta función recorre la lista que le pasemos y va soltando un componente CardPelicula por cada película. A cada tarjeta le pasamos sus datos, su clave única (key) y el aviso de si es favorita o no. Este último dato es el que decide si en la pantalla aparece el triángulo verde de "favorito" sobre la carátula.

- **Maquetado**

En el return definimos la estructura principal con un div y una etiqueta main que renderiza la variable contenido. Esta variable es dinámica y cambia según la interacción del usuario. Si se está realizando una búsqueda, muestra el componente ResultadosBusqueda. En caso contrario, carga el carrusel principal seguido de un Container que incluye el título de la sección, los selectores de navegación (Navs) y el listado de películas correspondiente al filtro que se haya pulsado.

2.2.4.6. TopPelículas

a) TopPelículas.tsx

- **Introducción**

En primer lugar, definimos las constantes para las medallas y sus colores que utilizaremos en el podio. Dentro del componente TopPelículas, inicializamos los hooks de traducción (t, i18n) y el estado para gestionar las películasRankeadas. También definimos un estado para el filtro, que nos permite controlar cuántos resultados queremos mostrar en pantalla según la elección del usuario.

- **useEffect:**

Al cargar el componente, ejecutamos un useEffect que obtiene los datos mediante la función getTopRanking del repositorio y aplica una lógica de segmentación sobre los resultados: primero, usamos un operador ternario con .slice(0, filtro) para limitar la cantidad de películas mostradas según el selector; después, extraemos los tres primeros elementos con .slice(0, 3) para asignarles el diseño especial de medallas y, finalmente, utilizamos .slice(3) para agrupar el resto de la lista en un formato más sencillo debajo del podio.

- **Maquetado:**

La estructura principal se define dentro de una etiqueta <main> que envuelve un Container de Bootstrap para centrar el contenido. En la cabecera, incluimos el título y subtítulo traducidos, junto a una botonera que permite al usuario filtrar dinámicamente el número de resultados (Top 5, Top 10 o todas). El cuerpo de la página se divide en dos secciones: primero, un podio visual para el Top 3, donde cada película se presenta en una tarjeta (top3-card) con su medalla correspondiente, imagen, categoría, nota media sobre 5 y una descripción limitada a 110 caracteres. Finalmente, para el resto de la lista, utilizamos un formato de fila más compacto que muestra la posición del ranking (empezando en el #4), la miniatura y un botón de acceso directo a los detalles de la película.

2.2.5. components

2.2.5.1. access

a) Login.tsx

En este componente gestionamos el acceso de los usuarios. La lógica se divide en los siguientes puntos:

- **Gestión de estados y Contexto:**

- **Datos del formulario:**

Utilizamos `useState` para controlar lo que el usuario escribe en el email y la contraseña en tiempo real.

- **Feedback visual:**

Hemos implementado un sistema de Modales (`MensajeModal`) para avisar al usuario si el login ha sido un éxito o si ha habido algún error (tal y como hemos hecho en otros casos)

- **Definición de estados:**

Para gestionar el formulario y la respuesta del sistema, definimos los siguientes estados mediante el Hook `useState`:

- **email y password:** Almacenan los datos que el usuario va escribiendo en los campos de texto del formulario.
 - **mostrarModal:** Es un booleano que controla si la ventana emergente de aviso está a la vista o escondida.
 - **tituloModal y mensajeModal:** Guardan los textos personalizados que queremos mostrar en el aviso (por ejemplo, "Error" o "Éxito").
 - **tipoModal:** Nos sirve para definir el estilo visual del aviso, diferenciando si es un mensaje de éxito (verde) o de error (rojo).

- **Funciones**

Se han definido las siguientes funciones:

- **lanzamientoAviso:**

Es la función que usamos para abrir los mensajes de alerta. Solo tenemos que pasarle el título y el texto que queremos mostrar, y ella se encarga de cambiar los estados del modal para que aparezca en pantalla.

- **submitHandler:**

Es la función que se activa al dar clic en el botón de enviar. Primero evita que la página se recargue sola, luego comprueba que el usuario haya escrito el email y la contraseña, y finalmente intenta hacer el login con los datos introducidos.

- **Control de errores:**

Si Firebase devuelve un fallo (por ejemplo, contraseña incorrecta), el bloque catch captura el código técnico. Gracias al AccessService, traducimos ese código a un mensaje que el usuario pueda entender y lo mostramos en el modal de error.

Podemos ver algunos de los errores con los que tratamos:

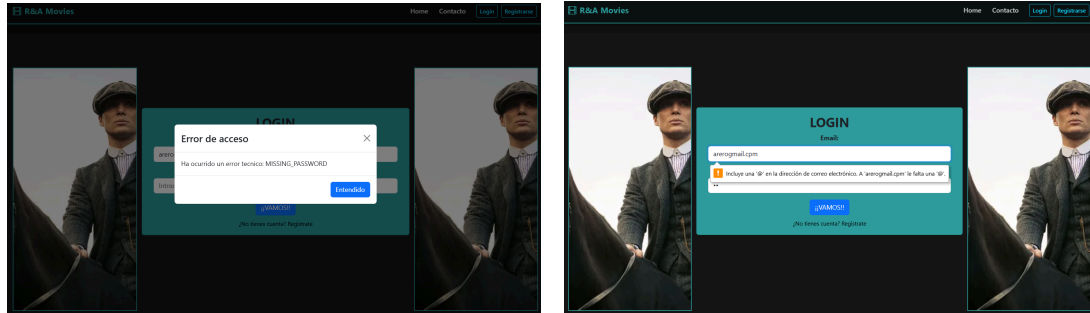


Fig 7. Error cuando no se introduce contraseña (1) y cuando no se introduce el correo de forma correcta (2)

- **Interfaz de usuario:**

La interfaz se ha diseñado utilizando el sistema de rejilla de React-Bootstrap, estructurando la página en un Container con una fila dividida en tres columnas. Las columnas laterales se han reservado para imágenes decorativas que equilibran visualmente el espacio, evitando que la página quede vacía.

En la columna central se ubica el núcleo: una Card que contiene el formulario. Dentro del Card.Body, se han organizado los campos de entrada mediante etiquetas Form.Group, asegurando un diseño profesional que facilita la interacción del usuario.

b) Registro.tsx

Este componente permite a los nuevos usuarios darse de alta en la plataforma. Su funcionamiento se resume en los siguientes puntos:

- **lógica del envío (mediante submitHandler)**

Al enviar el formulario, se ejecuta la función registroCompleto del repositorio. Esta función es clave porque realiza dos tareas en una: registra las credenciales en el sistema de autenticación de Firebase y, simultáneamente, crea un perfil de usuario en la base de datos con su nombre de usuario (username) y fecha de registro (tal y como hemos explicado previamente). Tras el éxito, se inicia sesión automáticamente y se redirige al usuario a la página de inicio tras un aviso de bienvenida.

- **Gestión de errores y feedback**

Al igual que en el login, el componente utiliza el AccessService para transformar los errores técnicos (como correos ya existentes o contraseñas débiles) en mensajes comprensibles. El estado de la operación se comunica a través del componente MensajeModal, que informa si el proceso ha sido satisfactorio o si ha ocurrido algún problema.

- **Interfaz de usuario**

La interfaz sigue la lógica establecida en el Login.tsx. En primer lugar establecemos un Container y tras ello definimos un <Row> dentro del cual establecemos 3 <Col>. Las columnas laterales se emplean para meter las imágenes que hagan que esta página no sea tan sosa. En medio se establece el una Card dentro de la cual se emplea el Form para poder introducir email, contraseña y nombre usuario

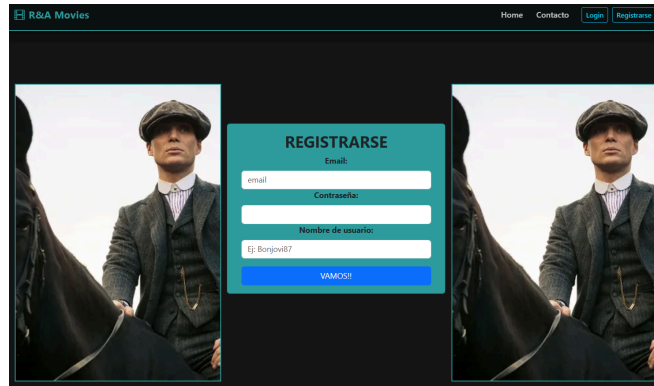


Fig 8. Visualización de la página de Registro

2.2.5.2. ui

a) CarouselPrincipal.tsx

El carrusel es el elemento principal de impacto en nuestra Home. Hemos decidido diseñarlo no como una simple sucesión de imágenes, sino como un expositor interactivo de Películas Destacadas.

- **Filtrado de datos:**

En lugar de mostrar todo el catálogo, utilizamos el método .filter para extraer únicamente las películas que tienen la propiedad destacada como true.

- **Estructura Multitarjeta:**

Hemos configurado dos tipos de visualización. Para pantallas grandes, el carrusel muestra grupos de tres tarjetas por diapositiva usando .slice(0, 3) y .slice(3, 6), aprovechando mejor el espacio. Para el formato móvil, el componente se adapta automáticamente para mostrar una sola tarjeta por vez.

- **Diseño de Tarjetas Horizontales:**

Mediante la función renderCard, creamos tarjetas personalizadas que dividen el contenido en dos columnas (Row y Col de Bootstrap). A la izquierda se muestra la tarjeta con object-fit: cover para que no se deforme, y a la derecha el cuerpo de la tarjeta con el título, la descripción y el botón de "Ver más" que enlaza directamente a la ficha de la película.

- **Control de Estados:**

Hemos utilizado el hook `useState` para controlar manualmente el índice del carrusel, asegurando que la transición entre los grupos de películas destacadas sea suave.

- **Gestión de Idiomas y Estado:**

El componente detecta el idioma activo (es, en, eu) para mostrar los textos correspondientes. Además, usamos el hook `useState` y la función `handleSelect` para controlar manualmente qué diapositiva se está mostrando, asegurando una navegación fluida.

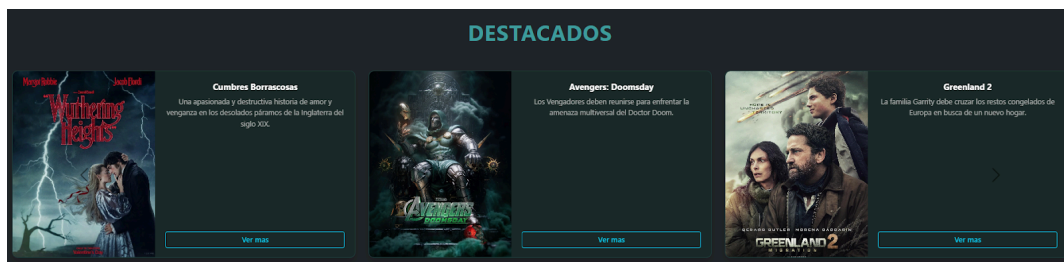


Fig 9. Visualización del apartado de Carrusel en la página principal

b) footer.css y Footer.tsx

El Footer (pie de página) ha sido diseñado para asentar la identidad visual de la aplicación y proporcionar al usuario enlaces de interés y datos de contacto de forma persistente en todas las rutas.

1) Footer.tsx

Hemos implementado las siguientes partes:

- **Estructura Multicolumna:**

Hemos empleado el sistema de rejilla de Bootstrap con `Row` y `Col`, dividiendo el contenido en cuatro bloques temáticos (Marca, Ubicación, Redes Sociales e Información legal). Gracias a las clases `xs`, `sm` y `md`, el diseño es totalmente responsivo, apilando las columnas en móviles y expandiéndolas en escritorio.

- **Navegación Interna y Externa:**

Hemos diferenciado el uso de etiquetas `<a>` para enlaces externos (como redes sociales) y el componente `<Link>` de `react-router-dom` para la navegación interna. Esto asegura que la aplicación no se recargue innecesariamente al navegar hacia páginas como "Aviso Legal".

- **Iconografía:**

Hemos integrado `bootstrap-icons` para que la interfaz sea más fácil de entender, añadiendo iconos visuales junto al nombre de la marca y las redes sociales.

2) footer.css

En cuanto al estilo, hemos buscado un acabado adecuado que contraste con el resto de la web:

- **Fondos:**

En lugar de un color plano, hemos usado un degradado (linear-gradient) entre tonos oscuros para dar sensación de profundidad.

- **Jerarquía visual:**

Los títulos de las columnas se han configurado en mayúsculas (uppercase) y con el color turquesa para que resalten. Para los textos informativos y enlaces, usamos un blanco con opacidad baja para que no cansen la vista y se entienda que son secundarios.

- **Interactividad (Hover):**

Los enlaces cambian de color suavemente mediante una transición de 0.2s.

- **Cierre:**

Hemos añadido una sección final (.footer-copy) para los derechos de autor.

c) Header.tsx y header.css

El Header actúa como el eje central de la experiencia de usuario. Siguiendo la metodología de trabajo del proyecto, hemos mantenido una separación clara entre la lógica del componente (.tsx) y la definición de sus estilos visuales (.css)

1) Header.tsx

- **Gestión del contexto:**

Utilizamos el hook useContext conectado a AuthContext para determinar si hay un usuario logueado. Dependiendo de este estado, la variable contenidoDerecha renderiza condicionalmente los botones de Login/Registro o el botón de Cerrar Sesión.

- **Parte del buscador:**

El input de búsqueda utiliza el evento onChange para disparar la función manejo_de_búsqueda. Esta función tiene una lógica inteligente: si el usuario empieza a escribir mientras está en una página secundaria (como "Contacto"), la aplicación le redirige automáticamente a la Home (/) para mostrarle allí los resultados

- **Tratamiento con múltiples idiomas:**

Implementamos un menú desplegable (NavDropdown) que permite cambiar el idioma de la interfaz entre Castellano, Inglés y Euskera. Al seleccionar una opción, se ejecuta i18n.changeLanguage, actualizando todos los textos de la web

- **Responsividad**

Gracias al componente <NavbarCollapse>, el menú es totalmente responsivo (se convierte en un menú "hamburguesa" en móviles).

2) header.css

- **Barra Fija:**

Le hemos puesto `sticky="top"` para que la barra se quede siempre arriba aunque hagas scroll hacia abajo. Tiene un color negro casi total pero con un pelín de transparencia.

- **Colores de Marca:**

El logo y los bordes usan nuestro verde turquesa. Es el color que identifica a "R&A Movies".

- **Efecto al pasar el ratón:**

Tanto el logo como los botones de menú cambian de color suavemente cuando pasas el cursor por encima (hover), así el usuario sabe que puede clicar ahí.

- **Sombra y Separación:**

Hemos puesto una sombra bajo la barra (box-shadow) para que parezca que está por encima del resto del contenido y un poco de separación entre botones para que no se vean pegados.

d) MensajeModal.tsx

Este componente se encarga de mostrar las notificaciones y errores de la web de forma visual. Para ello, utilizamos primero el hook `useTranslation` para que los avisos aparezcan en el idioma correcto (ES, EN o EU). La estructura del mensaje se divide en tres partes de Bootstrap: un `Modal.Header` para el título (como "Error" o "Éxito"), un `Modal.Body` para el texto del mensaje y un `Modal.Footer` donde incluimos el botón para cerrar la ventana. Así, el usuario recibe un aviso claro cada vez que intenta loguearse o registrarse.

2.2.6. store

2.2.6.1. AuthContext.tsx

Este archivo crea un almacén central de datos para la seguridad. Sirve para que cualquier parte de la aplicación sepa quién es el usuario sin tener que preguntarlo constantemente.

- **Datos guardados:**

Hemos definido que el sistema debe recordar siempre el estado de la sesión, el nombre del usuario, su código único de identificación y el token de seguridad para hablar con la base de datos.

- **Funciones de control:**

El contexto incluye las órdenes para entrar y salir de la cuenta. Gracias a esto, cuando se pulsa el botón de cerrar sesión, el cambio se nota en toda la web al instante.

- **Seguridad inicial:**

Por defecto, el sistema arranca con todo vacío y la sesión cerrada. Así nos aseguramos de que nadie vea datos privados sin identificarse primero.

2.2.7. utils

En esta carpeta hemos centralizado aquellas funciones de ayuda que se utilizan en diferentes partes de la web. Son herramientas auxiliares que no pertenecen a la lógica de negocio.

La función principal que se encuentra aquí es `renderEstrellas`. Su objetivo es transformar una nota numérica (por ejemplo, un 8) en un conjunto de iconos visuales de estrellas (de 0 a 5).

Lógica de la función

Dado que las puntuaciones en nuestra base de datos suelen estar en una escala sobre 10, hemos programado que la función divida la nota entre 2 para adaptarla al sistema visual de 5 estrellas.

Mediante un bucle `for` que recorre las 5 posiciones posibles, la función decide qué tipo de icono de Bootstrap Icons debe añadir al array final según el valor procesado:

- **Estrella rellena:** Si el número actual es menor o igual a la nota ajustada.
- **Media estrella :** Si la nota contiene un decimal de 0.5.
- **Estrella vacía :** Para completar el resto de posiciones hasta llegar a las 5 estrellas.

Finalmente, la función devuelve un elemento de React con todos los iconos agrupados.

3. Modelo de datos

3.1. Firebase Authentication

Este servicio se encarga exclusivamente de la seguridad y el acceso. Es donde se guardan los correos y las contraseñas de forma cifrada.

Cuando un usuario se registra o hace login, Authentication comprueba que los datos son correctos y nos devuelve un "token" (una clave de acceso).

3.2. Estructura de la Realtime Database

3.2.1. Nodo de películas

Es el catálogo central. Cada película tiene un número identificador y dentro guardamos toda su información: el título, la categoría, la sinopsis y las rutas de las imágenes de portada y fondo. También incluye detalles como si es un estreno próximo o la URL del vídeo local.

Toda la información que se almacena para cada película es la que se indica a continuación:

propiedad		Explicación
id		Identificador único de la película en la base de datos
destacada		Booleano que determina si la película aparece en el carrusel de la página principal.
calificacion_media		Valor numérico que representa la puntuación media obtenida por la película.
imagen_portada		URL o ruta de acceso a la imagen utilizada en las tarjetas y listados.
imagen_en_pelicula		URL de la imagen de fondo o cabecera utilizada en la vista de detalle.
video_local		Ruta al archivo de vídeo o enlace al tráiler promocional.
fecha_estreno		Fecha de lanzamiento oficial de la peli.
proximamente		Booleano para distinguir entre estrenos actuales y futuros lanzamientos.
taquilla		Cifra de recaudación económica obtenida por la película a nivel global.
pais_origen		Territorio o nación donde se realizó la producción principal.
mercados		Regiones geográficas donde la película tiene licencia de distribución.
es en eu	titulo	Nombre de la película (según idioma)
	categoria	Categoría a la que pertenece la película
	descripcion	Sinopsis o resumen de la trama en el idioma correspondiente.

Tabla 19. Estructura del modelo de datos para el objeto Película, detallando las propiedades técnicas

3.2.2. Nodo de usuarios

Aquí se almacena el perfil de cada persona que se registra. Guardamos su nombre, email y fecha en la que se creó la cuenta. Además, dentro de cada usuario tenemos un apartado de favoritos, donde se van añadiendo las IDs de las películas que el usuario ha decidido guardar.

propiedad		Explicación
OoOXXXXXX	email	Correo electrónico vinculada a la cuenta para el inicio de sesión.
	fecha_registro	Fecha exacta en la que el usuario creó su perfil en la aplicación.
	user	Nombre de usuario o alias elegido para mostrarse en la interfaz.

Tabla 20. Estructura del modelo de datos para el objeto usuarios

3.2.3. Nodo de puntuaciones

Es donde se registran las notas numéricas. Cada vez que un usuario puntúa, se guarda su ID, la ID de la película y la nota que ha puesto. Gracias a que estos datos están separados, la aplicación puede leer todas las notas de una película y calcular la media que se muestra en pantalla.

propiedad		Explicación
-XOOXXXX-	nota	Valor numérico que representa la puntuación otorgada.
	pelicula_id	Referencia (ID) de la película que está siendo valorada.
	usuario_id	Referencia (ID) del usuario que ha realizado la votación.

Tabla 21. Estructura del modelo de datos para el objeto puntuaciones

3.2.4. Nodo de comentarios

En este bloque se guardan todas las reseñas de la comunidad. Cada comentario es un objeto independiente que registra qué usuario lo ha escrito, en qué película lo ha puesto, el texto del mensaje y la fecha. Esto nos permite cargar el hilo de mensajes en la ficha de cada película.

propiedad		Explicación
-XXaXX-XXX	fecha	Marca de tiempo que indica cuándo se publicó la reseña
	pelicula_id	Referencia (ID) de la película que está siendo valorada.
	texto	Contenido alfanumérico que recoge la opinión o crítica escrita por el usuario.
	usuario_id	Referencia (ID) del usuario que ha realizado la votación.

Tabla 22. Estructura del modelo de datos para el objeto comentarios

4. Flujo de usuario y funcionalidades

4.1. Gestión de sesiones (access)

Se ha implementado un sistema para que los usuarios puedan crear su cuenta y entrar de forma segura. La sesión no se pierde al navegar; el sistema reconoce quién ha entrado y recupera su nombre y sus datos personales de la base de datos para que la experiencia sea personalizada en toda la web.

4.2. Experiencia de visualización

La web permite explorar el catálogo cómodamente. Se han organizado las películas para que se vea claramente que hay en cartelera y que llegarán próximamente. Al pinchar en una de ellas, la aplicación carga al mismo tiempo la sinopsis, los videos y las opiniones de otros usuarios sobre dicha peli.

4.3. Interactividad

Nosotros, tal y como nos pedían, queríamos que la web fuera una comunidad. Por ello, si estás logueado, puedes poner una nota entre 1 y 10 y escribir tus propios comentarios. El sistema suma todas las notas y calcula la media al momento, así que la valoración de la película cambia según lo que opina la gente.

4.4. Personalización

Se ha añadido una lista de favoritos para que cada uno pueda guardar lo que quiere ver. Además, para que sea más visual, se ha colocado un aviso visual en la página principal. Si una película ya está en tu lista, aparece una marca en la carátula para que el usuario lo sepa.

5. Interfaz y experiencia de usuario

5.1. Diseño visual

Se ha optado por emplear un diseño de modo oscuro porque es lo que mejor funciona para el caso del cine. Los colores principales son el gris oscuro y un turquesa para las cosas importantes, así la web se ve moderna.

5.2. Componentes globales

Para que la navegación sea fácil, el menú de arriba cambia solo. Si no has entrado, se te invita a Registrarte y si ya estás dentro, te permite Cerrar sesión. También hemos colocado un carrusel de destacados al principio para que la página tenga más movimiento.

5.3. Feedback y modales

No nos gustaban los mensajes típicos del navegador porque quedan como poco profesionales. Por ello, lo que hemos hecho ha sido crear nuestras propias ventanas de aviso que aparecen cuando se añade una peli a favoritos o cuando hay algún tipo de error.

6. Formato móvil (Responsive)

Cabecera

La idea básica que hemos utilizado en este caso ha sido la de emplear el `expand="md"` en la etiqueta `Navbar` que compacta todo. De esta forma conseguimos que, cuando estamos en formato móvil o más pequeño, se pase a esconder todo lo que hay en el Header dentro del botón de formato hamburguesa.

Por otro lado, conseguimos que todo aparezca alineado gracias al uso de las clases de utilidad de Flexbox de Bootstrap, como `ms-auto` y `align-items-center`. Estas clases permiten que los elementos de navegación y los botones de acceso se reorganicen verticalmente en móviles.

Finalmente, hemos integrado el buscador responsivo con un ancho máximo controlado

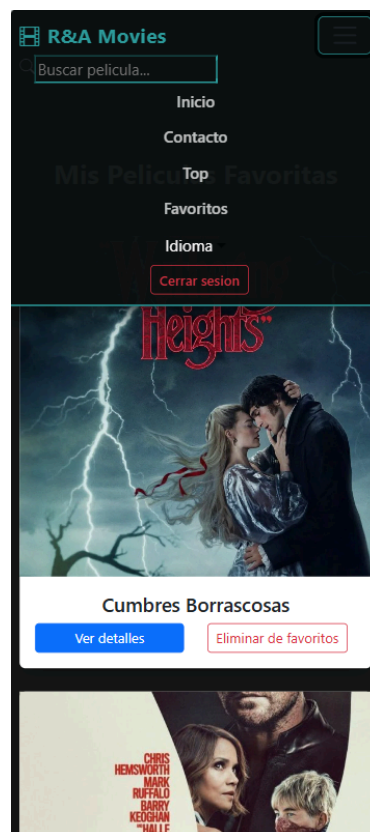


Fig 10. Captura de página cualquiera con visualización de cabecera

Página de la película en específico

Para adaptarnos en esta página, lo que hemos hecho ha sido aprovechar el sistema de rejilla de Bootstrap. Al configurar las columnas con `lg={7}` y `lg={5}`, conseguimos que en el ordenador el tráiler y la sinopsis se vean uno al lado del otro, pero en el móvil se apilen automáticamente para que el contenido ocupe todo el ancho de la pantalla y sea fácil de leer.

Además, para que los elementos no se amontonen, hemos aplicado clases de espaciado como gy-5 y flex-wrap. Esto hace que, si la información de la película no cabe en una sola línea en el móvil, salte a la siguiente.

Por último, para el reproductor de vídeo, hemos utilizado la clase ratio-16x9. Con esto nos aseguramos de que el tráiler mantenga siempre sus proporciones originales en cualquier dispositivo, evitando que la imagen se deforme

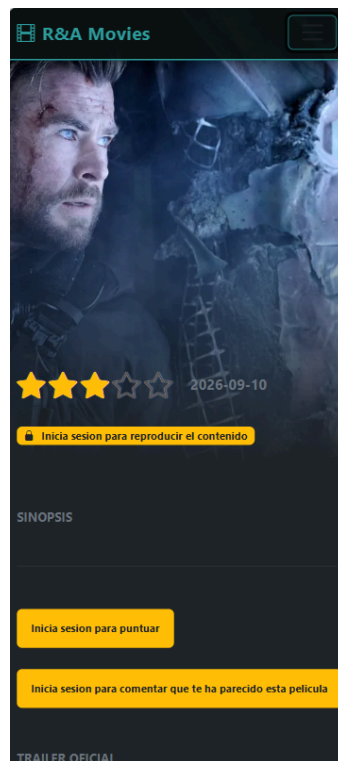


Fig 11. Captura de pagina de película en específico en formato móvil

Página de Top

En primer lugar lo que hemos hecho, para el apartado del top, es colocar en la columna que engloba a todo, xs de 12 que nos permite que esa película ocupe todo el ancho del móvil.

Además, para adaptarnos en esta página, lo que hemos hecho ha sido limitar la descripción de las películas con substring(0, 110). Esto nos permite controlar que el texto no sea demasiado largo en pantallas pequeñas.

Finalmente, para el resto de la lista, hemos aplicado propiedades de Flexbox avanzado en el CSS. Hemos utilizado flex-shrink: 0 en las imágenes para que no se aplasten en móviles y la propiedad text-overflow: ellipsis en los títulos. De esta forma, si el nombre de una película es demasiado largo, el sistema lo corta automáticamente con puntos suspensivos.

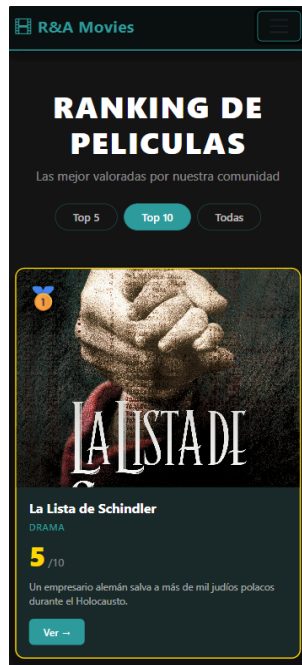


Fig 12. Captura de pagina de peliculas top en formato móvil

Página de contacto

Para adaptarnos en esta página, lo que hemos hecho ha sido configurar las tres cajas de información (email, teléfono y ubicación) con un $xs=\{12\}$. Esto permite que cada caja ocupe todo el ancho en formato móvil.

Por otro lado, hemos implementado en el MapContainer un ancho del 100%. Esto consigue que el mapa de las sedes sea totalmente fluido, estirándose o encogiéndose según el tamaño del dispositivo.

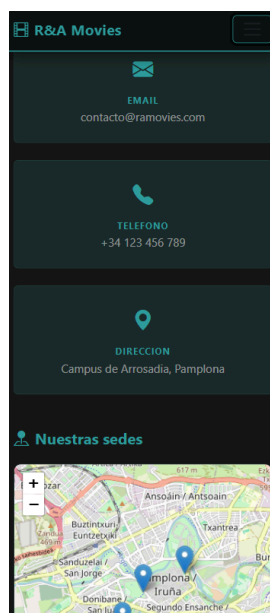


Fig 13. Captura de la página de contacto en formato móvil

Buscador

Para adaptarnos en esta sección, lo que hemos hecho ha sido envolver el listado de resultados en un Container y una Row de Bootstrap. Esto permite que las películas encontradas hereden el diseño responsivo de nuestras tarjetas, ajustando automáticamente el número de elementos por fila según el tamaño del dispositivo.

Además, para mejorar la experiencia en el móvil cuando no hay coincidencias, hemos configurado el mensaje de aviso con la clase w-100. De esta forma, el texto de "No hay resultados" aparece siempre centrado.

Finalmente, en la parte visual de los resultados, hemos aplicado la propiedad text-overflow: ellipsis a los títulos.

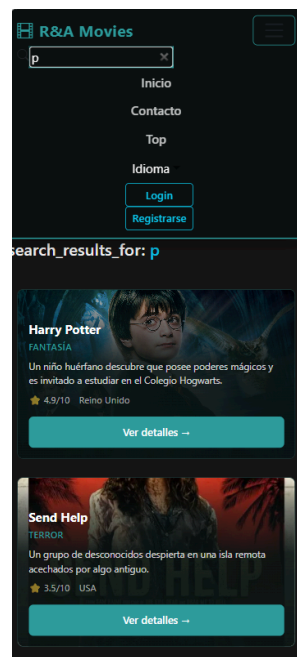


Fig 14. Captura del proceso de búsqueda para formato móvil

Favoritos

Para adaptarnos en esta página, lo que hemos hecho ha sido implementar dentro de la etiqueta Col el formato xs={12}. Esto permite que en dispositivos móviles cada tarjeta de película favorita se vea por sí sola ocupando todo el ancho de la pantalla.

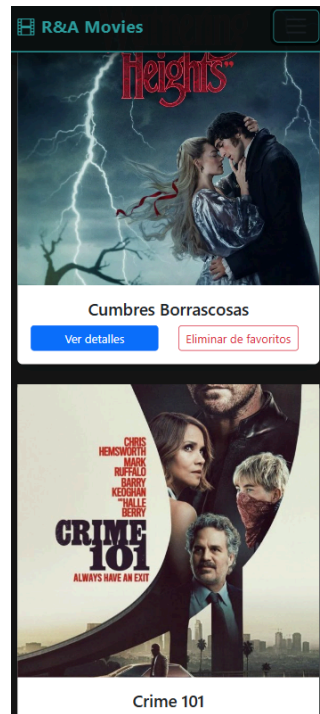


Fig 15. Captura de la página de favoritos para formato móvil

7. Mejoras introducidas para alcanzar los 2.5 puntos

7.1. Indicar las películas mejor valoradas

Para ofrecer un valor añadido al usuario, se ha desarrollado una página específica que rankea el contenido según su puntuación. Esta sección no solo lista las películas, sino que utiliza una jerarquía visual para destacar los títulos más exitosos. Para ello, nos hemos basado en dos pilares:

Lógica de negocio

El componente TopPelículas gestiona la información siguiendo estos principios:

- **Arquitectura Hexagonal:**
El componente no sabe de dónde vienen los datos; simplemente llama a `PeliculaRepository.getTopRanking()`, que se encarga de obtener y ordenar las películas por nota desde la infraestructura.
- **Gestión de Podio:**
Para dar más peso visual a los ganadores, dividimos la lista en dos grupos mediante el método `slice()`: un Top 3 (que recibirá un diseño de tarjetas especiales con medallas de oro, plata y bronce) y el resto del ranking (mostrado en un formato de lista más compacto).
- **Filtrado Dinámico:**
Se ha implementado un estado (filtro) que permite al usuario conmutar entre ver las 5 mejores, las 10 mejores o el catálogo completo, recalculando la vista al instante.
- **Soporte Multidioma:**
El componente gestiona la visualización de datos de forma dinámica detectando el idioma activo (es, en o eu) seleccionado por el usuario.

Diseño y Estética Visual (CSS)

El diseño se ha cuidado para que sea cinematográfico:

- **Personalización por Medallas:**
Se utilizan variables de CSS (`--medal-color`) para aplicar colores dinámicos (dorado, plata y bronce) a las tarjetas del podio según su posición.
- **Interactividad (Hovers):**
Las tarjetas del Top 3 cuentan con efectos de elevación (`translateY`) y zoom en la imagen, mientras que los elementos de la lista cambian de color de fondo para indicar que son interactivos.
- **Layout Responsivo:**
Gracias a React-Bootstrap, el podio se adapta de 3 columnas en pantallas grandes a una sola columna en dispositivos móviles, asegurando que la experiencia sea buena en cualquier dispositivo.

Veamos a continuación algún ejemplo de como queda esta parte de la aplicación:

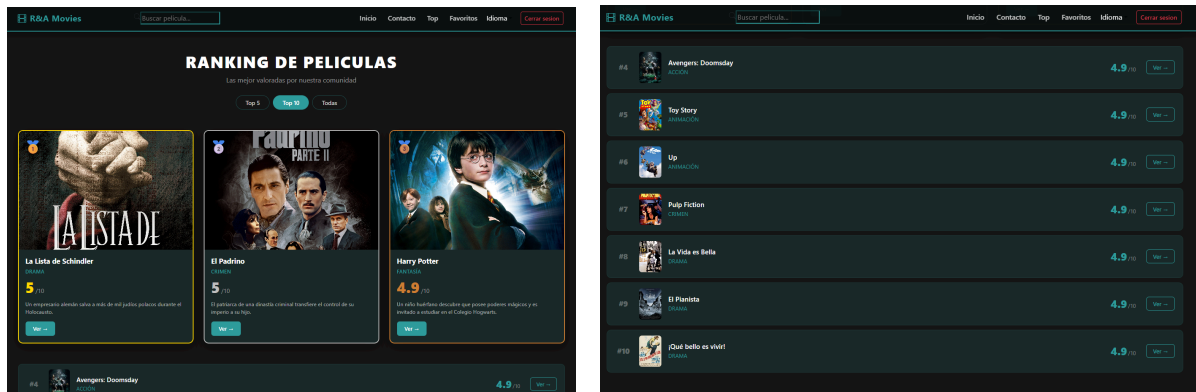


Fig 16. Diferentes vistas de la página Top

7.2. Mejores notificaciones que un simple alert

En lugar de utilizar los mensajes nativos del navegador (alert), que resultan poco profesionales y bloquean la navegación del usuario, hemos desarrollado un componente de avisos propio denominado MensajeModal.

Este componente se encuentra ubicado en la ruta src/components/ui/MensajeModal.tsx. Su funcionamiento se basa en una función que recibe un objeto de propiedades (AvisoDeProps), el cual define el comportamiento y contenido del aviso:

	Explicación
show	Un valor booleano que indica si el mensaje debe estar visible o no.
onHide	La función encargada de cerrar el modal cuando el usuario interactúa.
título	El encabezado que aparece en la parte superior del aviso.
mensaje	El cuerpo de texto con la información detallada.
tipo	Permite diferenciar visualmente si se trata de un mensaje de éxito (success) o de un error.

Tabla 23. parámetros del interfaz AvisoDeProps para el mensaje Modal generado

Para la interfaz, hemos utilizado la librería React Bootstrap empleando la etiqueta <Modal>. A esta etiqueta le pasamos las propiedades de control de estado y el atributo centered, que asegura que el mensaje aparezca siempre en el centro.

La estructura interna del componente se organiza en tres secciones de React Bootstrap: un Modal.Header que incluye el título, un Modal.Body donde se visualiza el cuerpo del mensaje, y un Modal.Footer que contiene el botón de confirmación ("Entendido"), permitiendo al usuario cerrar la ventana tras leer el aviso.

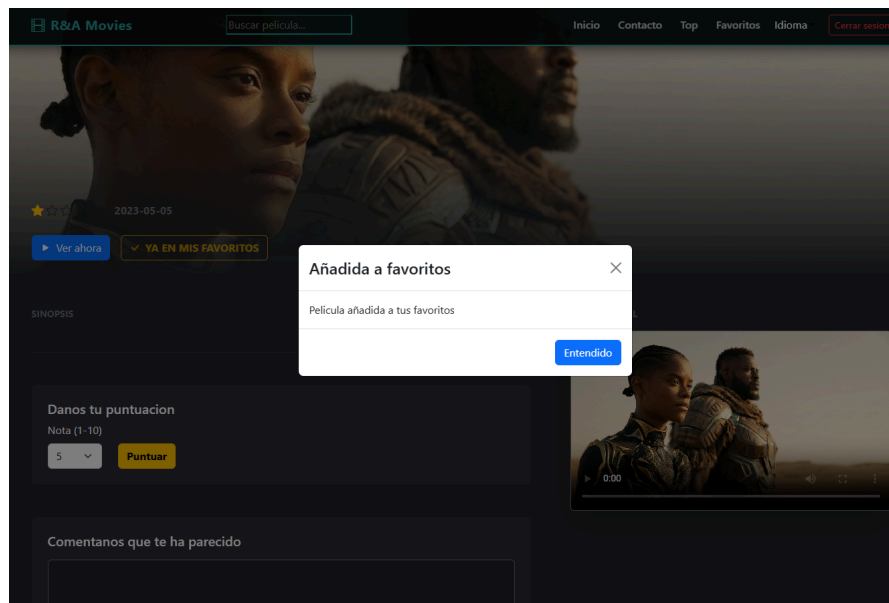


Fig 17. Visualización de mensaje indicativo de que se añadido película a favoritos

7.3. Posibilidad de implementar diferentes idiomas

Para poder implementar esta funcionalidad, se ha integrado la librería i18next junto con su adaptador para React (react-i18next).

La implementación se ha estructurado en tres niveles:

- **Centralización de traducciones:**

Se ha creado un archivo de configuración (i18n.ts) donde se inicializa la librería. Aquí se definen los idiomas disponibles (Castellano, Inglés y Euskera) y se cargan los diccionarios de recursos (archivos JSON) que contienen las equivalencias de cada texto de la interfaz.

- **Uso de Translation Keys:**

En lugar de escribir texto estático en los componentes, se utiliza el hook useTranslation. Este permite sustituir cadenas de texto por llaves de traducción, por ejemplo:

```
<span>{t('home_tab_billboard')}</span>
```

- **Filtrado de datos según disponibilidad:**

Para garantizar una buena experiencia de usuario, el repositorio de películas incluye una lógica de validación que asegura que solo se muestren en la plataforma aquellas películas que tengan contenido cargado en, al menos, uno de los idiomas soportados.

Podemos ver a continuación como se ve la pagina principal de la web en los tres idiomas permitidos (Castellano, inglés y euskera):

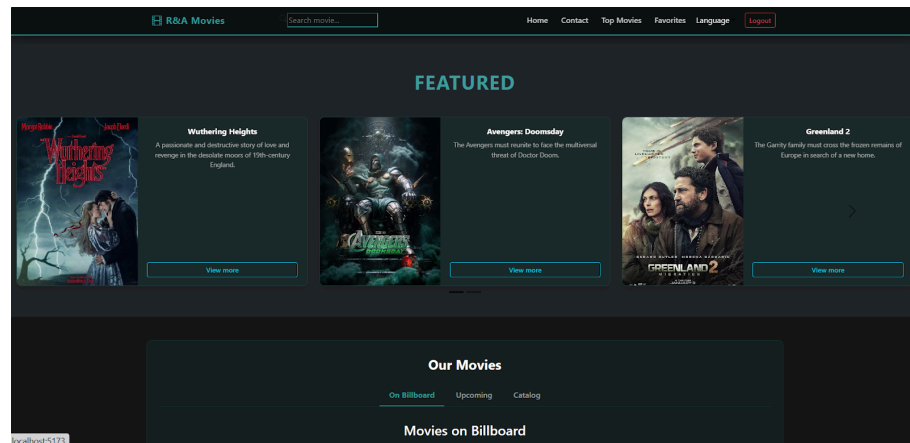


Fig 18. Interfaz de usuario en inglés

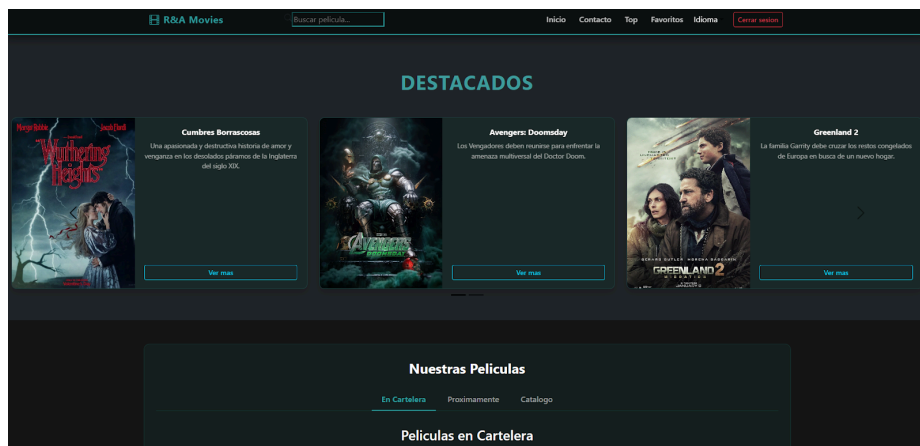


Fig 19. Interfaz de usuario en Castellano

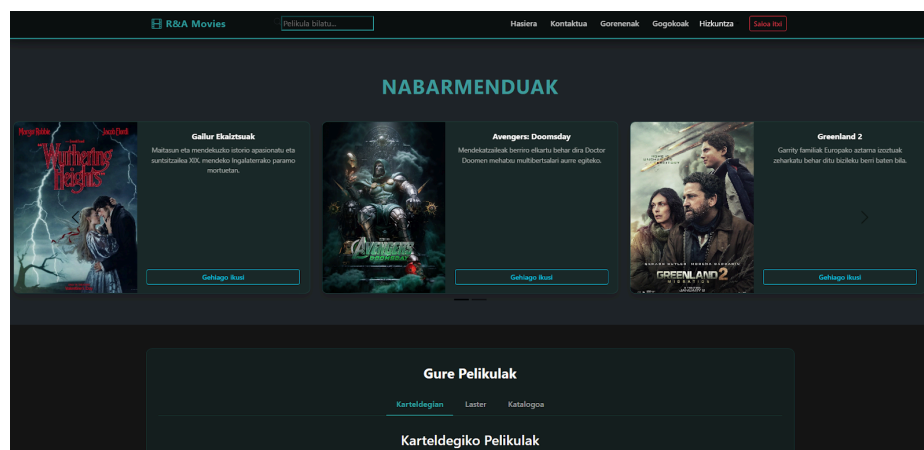


Fig 20. Interfaz de usuario en euskera

7.4. Uso de librería para introducir mapa

Aunque nuestra plataforma es principalmente un servicio digital de catálogo de películas, hemos decidido integrar un mapa con sedes físicas para dar una imagen de empresa más real y fiable. Estos puntos no solo representan nuestras oficinas centrales de soporte, sino que están pensados como espacios estratégicos donde los usuarios podrían recoger ediciones especiales de merchandising físico o asistir a preestrenos y eventos publicitarios organizados por la propia web.

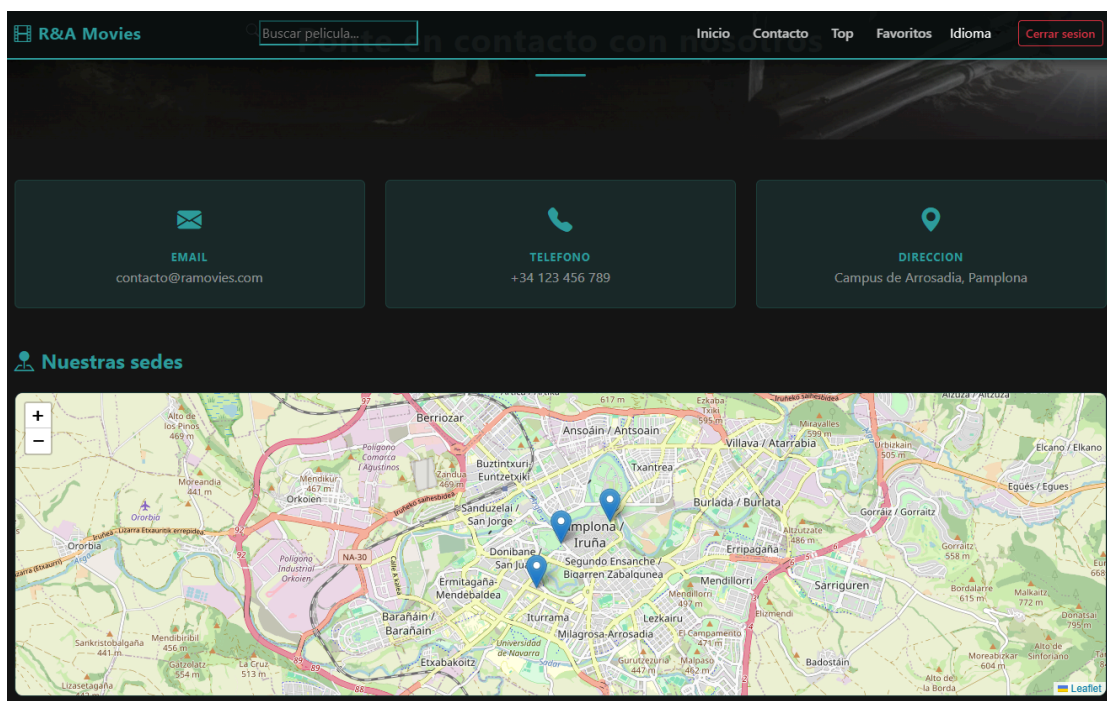


Fig 21. Interfaz del mapa interactivo con marcadores de sedes

Detalles de la implementación

Para llevar a cabo esta funcionalidad, se ha utilizado la librería React-Leaflet, una solución basada en OpenStreetMap que permite insertar mapas interactivos. La implementación se ha realizado siguiendo estos puntos:

- **Definición de coordenadas:**
Se ha establecido un punto de enfoque central (Pamplona) con un nivel de zoom optimizado para visualizar todas las sedes de un vistazo.
- **Gestión de marcadores:**
Se ha desarrollado un listado de objetos que contiene la latitud y longitud de cada oficina. Mediante un renderizado iterativo, se ha automatizado la colocación de marcas en cada ubicación geográfica.
- **Interactividad mediante ventanas emergentes:**
Se han configurado elementos de tipo Popup, de modo que, al interactuar con cualquier marcador, se muestra el nombre específico de la sede consultada.

7.5. Implementación de triángulo + estrella para indicar película favorita

Como ya se ha detallado en el apartado de Diseño Visual (Ribbon), hemos utilizado una combinación de CSS y componentes de Bootstrap para marcar las películas favoritas.

En lugar de un simple icono, hemos optado por un indicador visual en la esquina (Ribbon) que incluye una estrella dorada sobre un fondo triangular. Esta implementación permite que el usuario identifique de un solo vistazo qué contenidos están destacados o guardados en su lista personal

7.6. Implementación de búsqueda

Para mejorar la usabilidad de la plataforma, se ha desarrollado un componente encargado de mostrar las coincidencias según los términos introducidos por el usuario.

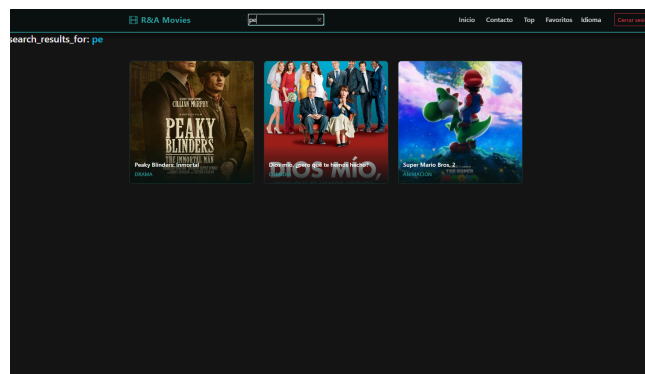


Fig 22. Ejemplo de cómo se visualiza la web cuando escribimos una búsqueda

La gestión de los resultados se ha estructurado en base a tres pilares:

1. Filtrado por idioma activo:

Dado que los títulos de las películas pueden variar entre castellano, inglés o euskera, se ha implementado una lógica que detecta el idioma del navegador mediante i18next.

2. Búsqueda insensible a mayúsculas:

Para evitar errores de usuario, se ha normalizado tanto el texto introducido como los títulos de la base de datos a minúsculas (toLowerCase()).

3. Gestión de estados vacíos:

En caso de que no existan coincidencias, se ha diseñado una vista de "No resultados" que informa, utilizando traducciones dinámicas para mostrar el término que ha fallado.

7.7. Opción de ver la película (que aparezca) aunque no se pueda poner por derechos

En este apartado explicamos cómo hemos desarrollado el componente "Modo Cine", ubicado en la ruta `components/peliculas/view/ModoCine.tsx`. Debido a que no es posible reproducir los archivos de vídeo reales por restricciones de copyright, hemos diseñado una interfaz que simula el entorno de visionado.

Implementación técnica del componente

Para la creación de este módulo, nos hemos basado en el uso de propiedades y una estructura visual sólida:

- **Interfaz de comunicación:**
El componente recibe los datos a través de `ModoCineProps`. Hemos definido dos parámetros de entrada fundamentales: `tituloPeli` (para identificar qué contenido se intenta ver) y `onClose` (la función que permite cerrar el reproductor).
- **Diseño del reproductor:**
Hemos establecido un contenedor con una relación de aspecto de 16:9 y fondo negro absoluto. Con esto, hemos conseguido recrear la estética de una sala de cine.
- **Control de interfaz:**
Para el botón de cierre, hemos aplicado un posicionamiento absoluto con un `zIndex`: 10. Esto nos asegura que el control de salida siempre esté por encima del reproductor y sea fácilmente accesible para el usuario.
- **Sistema de avisos:**
Como solución a la falta de derechos, hemos integrado etiquetas de tipo `Badge` de Bootstrap. Estos mensajes de error los hemos conectado a nuestro sistema de traducción (`i18next`) para que el usuario reciba la información en su idioma preferido.

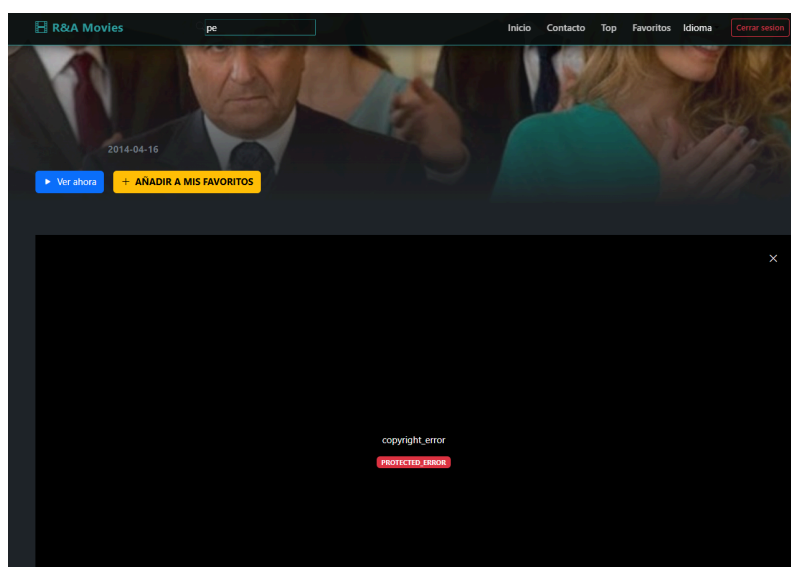


Fig 23. Interfaz del Modo Cine con aviso de protección de contenidos.

Flujo de navegación y experiencia de usuario (UX)

- **Activación:**
Al pulsar en el botón de reproducción, activamos este componente pasando dinámicamente el nombre de la película.
- **Cierre fluido:**
Al interactuar con el botón de cierre, ejecutamos la función `onClose`, lo que nos permite desmontar el componente y devolver al usuario a la vista de detalles sin interrupciones.
- **Responsividad:**
Al trabajar con estilos, hemos logrado que el reproductor mantenga sus proporciones en cualquier tamaño de pantalla, desde dispositivos móviles hasta monitores de escritorio.

7.8. Mejora en la visualización de las Cards de las películas

En este apartado describimos el desarrollo del componente `CardPelicula`. Hemos buscado un equilibrio entre estética y funcionalidad, permitiendo que el usuario obtenga información clave de la película simplemente pasando el cursor por encima.

Lógica de Internacionalización Dinámica

Uno de los mayores retos que hemos resuelto es cómo mostrar los datos en el idioma correcto dentro de la tarjeta:

- **Selección de idioma:**
Hemos configurado una constante `lang` que detecta el idioma activo del sistema (es, en, o eu).
- **Sistema de seguridad:**
Para evitar errores de renderizado si faltase alguna traducción, hemos implementado una lógica que prioriza el idioma seleccionado, pero recurre al castellano (`pele['es']`) por defecto.

Arquitectura de la Tarjeta (Efecto Netflix)

Para que el diseño sea impactante, hemos estructurado el HTML de la siguiente manera, conectándolo con nuestras clases personalizadas:

1. **Contenedor Base (`.netflix-card`):**
Es la caja principal que contiene la imagen de portada. Hemos aplicado una transición suave para que el zoom no sea brusco.
2. **Barra de Título Estática (`.netflix-title-bar`):**
Mientras la tarjeta está en reposo, hemos decidido mostrar solo el título y la categoría en la parte inferior sobre un degradado oscuro, garantizando la legibilidad.

3. Capa de Información (.netflix-hover-overlay):

Al activar el hover, esta capa aparece suavemente. En ella hemos incluido:

- **Resumen inteligente:**
Para no romper el diseño, hemos programado una función que corta la descripción a los 130 caracteres y añade puntos suspensivos si es necesario.
- **Metadatos:**
Hemos añadido iconos sutiles (como la estrella de calificación ★) y el país de origen para dar contexto rápido al usuario.
- **Enlace dinámico:**
Hemos integrado el componente Link de react-router-dom para que el botón "Ver detalles" redirija al usuario a la ficha técnica completa usando el id único de la película.

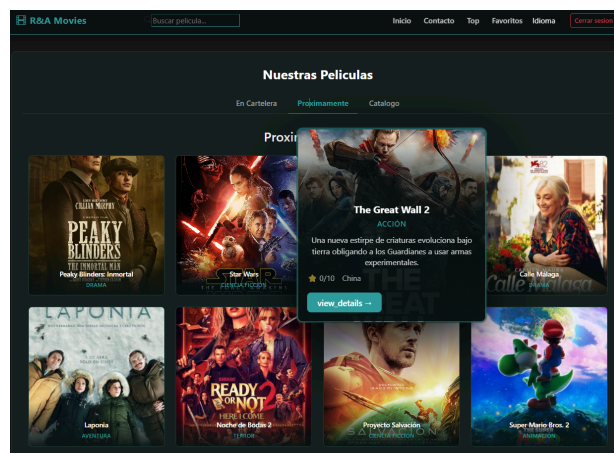


Fig 24. Estructura interna y capas del componente CardPelícula.

Adaptabilidad (Responsividad)

Nos hemos asegurado de que el grid sea totalmente responsive utilizando las columnas de Bootstrap en el componente Col. Hemos definido diferentes anchos según el dispositivo:

- Móvil: 1 tarjeta por fila (xs={12}).
- Tablet: 2 o 3 tarjetas (sm={6}, md={4}).
- Escritorio: 4 tarjetas por fila (lg={3}).

8. Metodología y Control de versiones

8.1. Gestión del repositorio

Se ha usado GitHub para guardar todo el código. Se ha trabajado separando las funciones nuevas en ramas distintas para no estropear lo que ya funcionaba bien. Antes de unir los cambios a lo que es la versión final, se revisan para comprobar que todo estuviera correctamente.

8.2. Historial de commits

Se han ido guardando cada avance de forma ordenada. Esto nos ha servido mucho, sobre todo para cambiar la estructura a hexagonal, ya que esto nos ha permitido mover archivos y reorganizarlos siempre teniendo una copia de seguridad de los pasos anteriores.

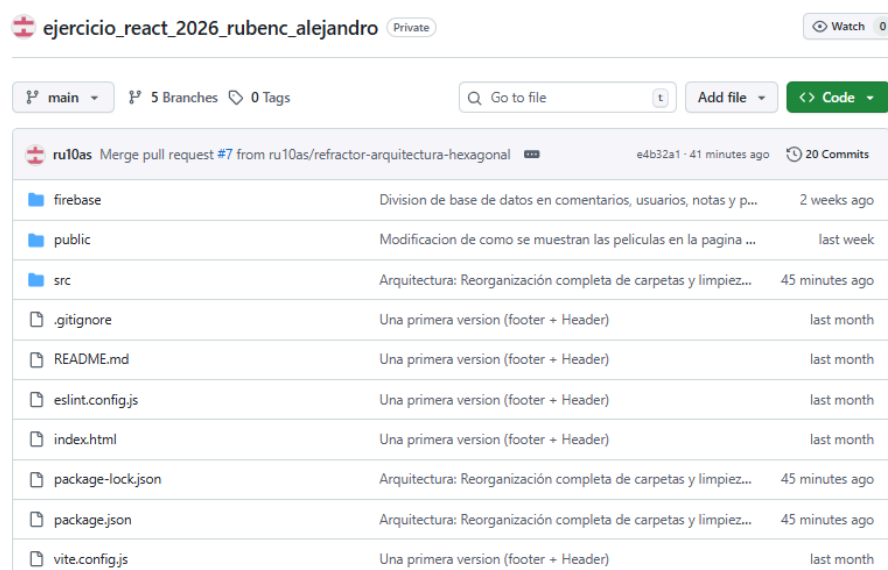


Fig 25. Visualización de como se ve en nuestro gitHub (en un paso intermedio)

8.3. Despliegue (Hosting)

La aplicación se ha subido a Firebase Hosting. Se ha conectado el código con el servidor para que, cada vez que terminamos una mejora importante, la web se actualice automáticamente.

Pasos seguidos:

- **Compilación**

```
npm run build
```

- **Preparación:**

Verificamos que los cambios locales se hayan guardado correctamente para evitar subir versiones obsoletas.

- **Despliegue:**

firebase deploy

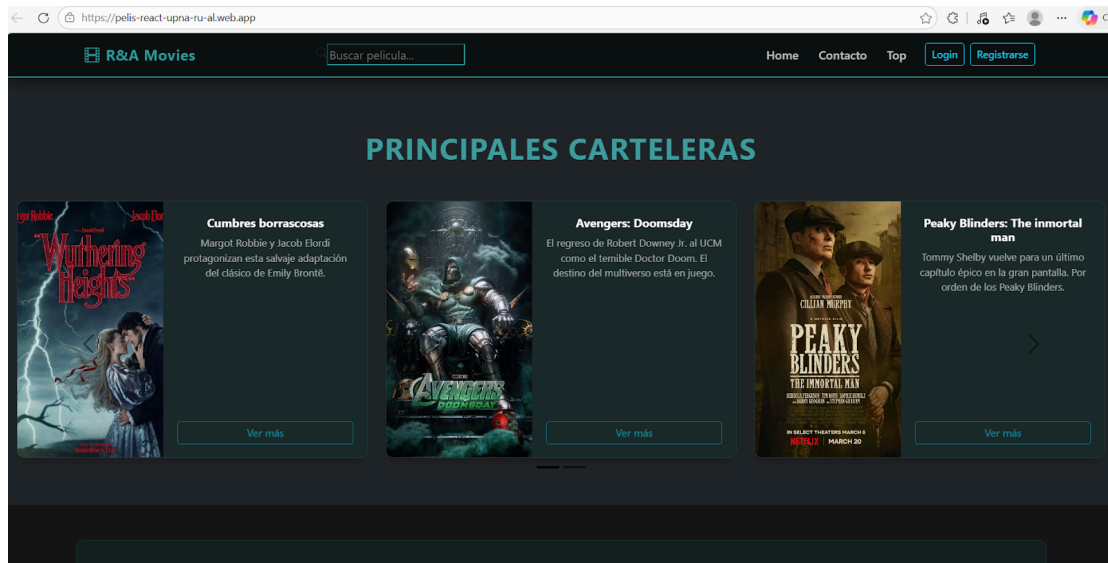


Fig 26. Visualización de como se ve nuestra página web

9. Consecución de los objetivos

Objetivos principales	
Objetivo	Consecución de dicho objetivo
En la página principal mostraremos un catálogo de películas	Hemos implementado una Home con un grid de tarjetas.
Pulsando cada película accederemos a una ficha con información detallada de la misma	Hemos desarrollado una vista específica (DetallePelícula) que muestra sinopsis, reparto y metadatos técnicos de cada título.
Crear dos páginas estáticas con, por ejemplo, información de contacto y aviso legal.	Hemos creado las secciones de Contacto y Aviso Legal con rutas independientes para cumplir con los estándares web.
Los usuarios registrados podrán puntuar las películas.	Hemos programado una lógica de estrellas para usuarios autenticados, vinculando cada voto al ID único del usuario.
Se deberá calcular la puntuación media para cada película y mostrar en su ficha.	Hemos implementado un servicio que recalcula la media aritmética de todas las valoraciones y la muestra en la cabecera de la ficha.
Los usuarios registrados también pueden hacer comentarios de las películas.	Hemos habilitado un área de texto para que los usuarios registrados puedan compartir su opinión
La ficha mostrará el hilo de comentarios	Hemos diseñado un componente que lista cronológicamente todos los comentarios realizados por la comunidad en cada película.
Los usuarios registrados también pueden añadir las películas de su gusto a sus Favoritos.	Hemos desarrollado la funcionalidad para añadir o quitar películas de la lista personal del usuario
En otra página, se podrá ver el listado de sus favoritos.	Hemos creado una vista exclusiva (MisFavoritos) que filtra y muestra solo las películas guardadas por el usuario actual.
Finalmente desplegamos el proyecto en el servicio de hosting de Firebase.	Hemos configurado y desplegado la aplicación en Firebase Hosting

Fig 27. Resumen de hitos alcanzados (1)

Procesos adicionales realizados	
Página con listado de las películas de mejor a peor valorada	Hemos creado una sección específica donde el catálogo se ordena automáticamente de mejor a peor valorada por la comunidad.
Mejores notificaciones	Hemos integrado avisos visuales (modales)
Implementación en varios idiomas	Hemos implementado traducciones completas en Castellano, Inglés y Euskera.
Uso de mapa	Hemos incluido un mapa interactivo en la sección de contacto para ubicar físicamente nuestras oficinas.
indicativo de que es una película favorita	Hemos diseñado un ribbon verde visual sobre la tarjeta de la película si esta ya se encuentra en la lista del usuario.
Posibilidad de buscar película en específico	Hemos desarrollado una lógica de búsqueda por texto que filtra el catálogo en tiempo real según el título de la película.
Opción de colocar el modo cine	Hemos implementado una funcionalidad exclusiva que adapta la interfaz para el visionado
Mejora en la visualización de las Cards de las películas	Hemos rediseñado las tarjetas con el "Efecto Netflix"

Fig 28. Resumen de hitos alcanzados (2)

10. Conclusiones y posibles mejoras

Tras finalizar el desarrollo de esta plataforma de streaming, hemos llegado a la conclusión de que la combinación de React con una Arquitectura Hexagonal nos ha permitido crear una aplicación robusta, escalable y con una interfaz de usuario de alto nivel. Hemos logrado hitos técnicos importantes, como la gestión de datos en tiempo real y, sobre todo, la implementación de un sistema de interactividad donde las valoraciones y reseñas actualizan automáticamente la calificación media de cada película.

Como equipo, hemos identificado las siguientes vías para seguir evolucionando el proyecto:

Gestión de Derechos y Streaming Real

Si quisiéramos ir a más, podríamos establecer los derechos de emisión y distribución digital mediante la implementación de un servidor de medios especializado. Esto requeriría integrar protocolos de transferencia de datos por paquetes (como flujo adaptativo), permitiendo que la calidad del vídeo se ajuste automáticamente a la conexión del usuario y garantice un visionado fluido sin interrupciones.

Algoritmo de Recomendación Inteligente

Una vez que hemos consolidado el sistema de valoraciones, una mejora de alto nivel sería desarrollar un motor de sugerencias. Este algoritmo analizaría las categorías y puntuaciones que el usuario suele dar para proponerle automáticamente nuevos títulos del catálogo que encajen con sus gustos personales.

Optimización para Dispositivos

Por último, hemos valorado la transformación de la web en una aplicación progresiva. Con esta mejora, conseguiríamos que la plataforma fuera instalable en cualquier dispositivo, ofreciendo tiempos de carga casi instantáneos y permitiendo la consulta del catálogo básico incluso en situaciones de baja conectividad a internet.